```
# WatsonxLLM
model = ModelInference(
   model_id=model_id,
   params=parameters,
   credentials=credentials,
   project_id=project_id
)

llm = WatsonxLLM(model=model)
response = llm.invoke("your prompt")



# Message Types
msg = llm.invoke([
   SystemMessage(content="system instruction"),
   HumanMessage(content="user message"),
   AIMessage(content="previous ai response")
])



# PromptTemplate
prompt = PromptTemplate.from_template(
   "Tell me a {adjective} joke about {topic}"
)

formatted_prompt = prompt.invoke({
   "adjective": "funny",
   "topic": "cats"
```

```
})


# ChatPromptTemplate
prompt = ChatPromptTemplate.from_messages([
   ("system", "You are helpful"),
   ("user", "Tell me about {topic}")
])

formatted_messages = prompt.invoke({
   "topic": "AI"
})


# MessagesPlaceholder
prompt = ChatPromptTemplate.from_messages([
   ("system", "You are helpful"),
   MessagesPlaceholder("msgs")
])

formatted_messages = prompt.invoke({
   "msgs": [HumanMessage(content="Hello")]
})


# JsonOutputParser
parser = JsonOutputParser(pydantic_object=Schema)
```

```
chain = prompt | llm | parser

result = chain.invoke({
   "query": "your query"
})


# CommaSeparatedListOutputParser
parser = CommaSeparatedListOutputParser()

chain = prompt | llm | parser

result = chain.invoke({
   "subject": "ice cream flavors"
})


# Document
doc = Document(
   page_content="your text",
   metadata={"source": "data"}
)


# PyPDFLoader
loader = PyPDFLoader("file.pdf")
documents = loader.load()
```

```
# WebBaseLoader
loader = WebBaseLoader("https://example.com")
documents = loader.load()



# CharacterTextSplitter
splitter = CharacterTextSplitter(
   chunk_size=200,
   chunk_overlap=20
)

chunks = splitter.split_documents(documents)



# RecursiveCharacterTextSplitter
splitter = RecursiveCharacterTextSplitter(
   chunk_size=500,
   chunk_overlap=50
)

chunks = splitter.split_documents(documents)



# WatsonxEmbeddings
embedding = WatsonxEmbeddings(
   model_id="embedding-model",
   url="url",
```

```
    project_id="project"
)


# Chroma
vectorstore = Chroma.from_documents(
    chunks,
    embedding
)

docs = vectorstore.similarity_search("query")


# Retriever
retriever = vectorstore.as_retriever()

docs = retriever.invoke("query")


# ParentDocumentRetriever
retriever = ParentDocumentRetriever(
    vectorstore=vectorstore,
    docstore=store,
    child_splitter=child_splitter,
    parent_splitter=parent_splitter
)

retriever.add_documents(documents)
```

```
docs = retriever.invoke("query")



# RetrievalQA
qa = RetrievalQA.from_chain_type(
  llm=llm,
  retriever=retriever
)

answer = qa.invoke("question")



# ChatMessageHistory
history = ChatMessageHistory()

history.add_user_message("Hello")
history.add_ai_message("Hi")

response = llm.invoke(history.messages)



# ConversationBufferMemory
conversation = ConversationChain(
  llm=llm,
  memory=ConversationBufferMemory()
)
```

```
response = conversation.invoke(
   input="Hello"
)



# LLMChain
chain = LLMChain(
   llm=llm,
   prompt=prompt
)

result = chain.invoke({
   "input": "data"
})



# SequentialChain
overall_chain = SequentialChain(
   chains=[chain1, chain2],
   input_variables=["input"],
   output_variables=["output"]
)

result = overall_chain.invoke({
   "input": "data"
})
```

```
# RunnablePassthrough
overall_chain = (
   RunnablePassthrough.assign(
      step=lambda x: chain.invoke(x)
   )
)

result = overall_chain.invoke({
   "input": "data"
})



# Tool
tool = Tool(
   name="Calculator",
   func=function_name,
   description="tool description"
)

result = tool.invoke("input")



# @tool decorator
@tool
def my_tool(data: str):
   return data
```

```
# create_react_agent
agent = create_react_agent(
   llm=llm,
   tools=tools,
   prompt=prompt
)



# AgentExecutor
executor = AgentExecutor(
   agent=agent,
   tools=tools
)

result = executor.invoke({
   "input": "question"
})
```